

ARIN5204: Reinforcement Learning

3.4 Continuous Controls

Dr. Long Chen

longchen@ust.hk

Assistant Professor, Dept. of CSE

The Hong Kong University of Science and Technology (HKUST)

May 5, 2026



- 1 Discrete vs. Continuous Control
- 2 Deterministic PG (DPG)
- 3 Deterministic PG Theorem
- 4 DDPG Implementation
- 5 Extension: TD3

Some slides are borrowed from: Pieter Abbeel (UCB), Shusen Wang (Meta).

① Discrete vs. Continuous Control

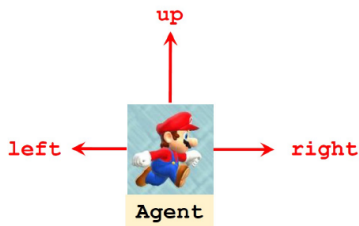
② Deterministic PG (DPG)

③ Deterministic PG Theorem

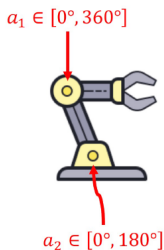
④ DDPG Implementation

⑤ Extension: TD3

Discrete vs. Continuous Action Space



- The action space A is **discrete**
- Action space $A = \{\text{left}, \text{right}, \text{up}\}$

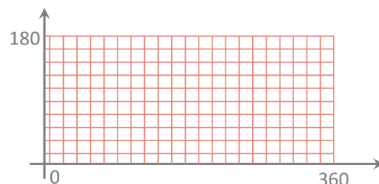


- Action space A is **continuous**

$$A = [0^\circ, 360^\circ] \times [0^\circ, 180^\circ]$$

- Actions are 2-dim vectors

Discretization



- **Straightforward Idea:** Discretize the action space. (Draw a grid.)
- Now, the number of actions is the number of grid points.
- Problem: [curse of dimensionality](#).
 - Let d be the degree of freedom
 - The number of actions grows exponentially with d

Stochastic Policy vs. Deterministic Policy

	Stochastic Policy	Deterministic Policy
Policy:	$\pi(a s;\boldsymbol{\theta})$	$\pi(s;\boldsymbol{\theta})$
Output:	Probability distribution over the action space	Action a
Control:	Randomly sample an action from the distribution	Directly use the output a
Applications:	Mostly discrete control	Continuous control

① Discrete vs. Continuous Control

② Deterministic PG (DPG)

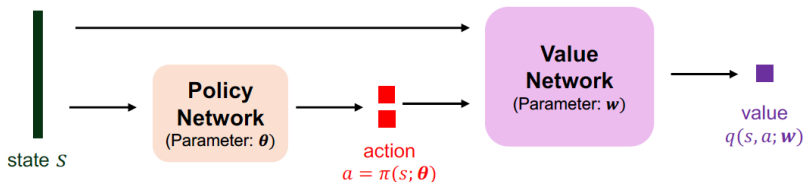
③ Deterministic PG Theorem

④ DDPG Implementation

⑤ Extension: TD3

Deterministic Actor-Critic

- Use a **deterministic** policy network (actor): $a = \pi(s; \theta)$
- Use a value network (critic): $Q(s, a; w)$
- The critic outputs a scalar that evaluates **how good the action** a is.



Policy Gradients So Far

- Policy objective:

$$\max_{\boldsymbol{\theta}} \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}(\tau)} [G(\tau)]$$

- New policy objective:

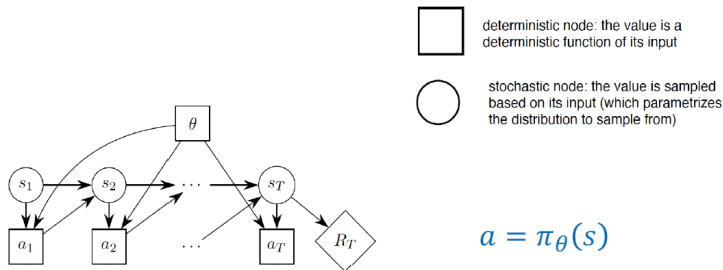
$$\max_{\boldsymbol{\theta}} \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}(\tau)} \left[\sum_{t=1}^T Q(s_t, a_t) \right]$$

- Q: Can we backpropagate through the Q function?
- If the policy is deterministic, we already know how to do it via the chain rule!

$$\mathbb{E} \left[\sum_t \frac{dQ(s_t, a_t)}{d\boldsymbol{\theta}} \right] = \mathbb{E} \left[\sum_t \frac{dQ(s_t, a_t)}{da_t} \cdot \frac{da_t}{d\boldsymbol{\theta}} \right]$$

Deep Deterministic Policy Gradients

Intuitive Understanding:



$$\mathbb{E} \left[\sum_t \frac{dQ(s_t, a_t)}{d\theta} \right] = \mathbb{E} \left[\sum_t \frac{dQ(s_t, a_t)}{da_t} \cdot \frac{da_t}{d\theta} \right]$$

① Discrete vs. Continuous Control

② Deterministic PG (DPG)

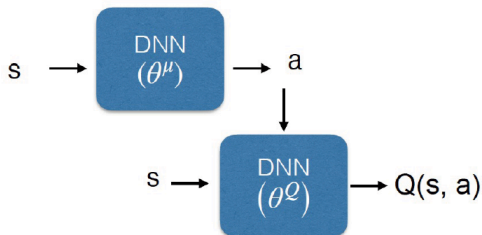
③ Deterministic PG Theorem

④ DDPG Implementation

⑤ Extension: TD3

Deep Deterministic Policy Gradients

The computational graph:



DDPG: Deep Q-Learning for continuous actions

Gradients of Deterministic Policies

Relation to Value-based RL & Policy-based RL:

- **Generalized policy iteration**: policy evaluation + policy improvements
- **Policy iteration**: The most common approach is a greedy maximization (or soft maximization) of the action-value function, $\mu_{k+1}(s) = \operatorname{argmax}_a Q_{\mu_k}(s, a)$
- **Continuous action spaces**:
 - Greedy policy improvement becomes **problematic**, requiring a global maximization at every step.
 - A simple and computationally attractive alternative: move the policy in the direction of the gradient of Q , rather than globally maximizing Q

Intuitive Idea of Deterministic Policies

- For each visited state s , the policy parameters θ_{k+1} are updated in proportion to the gradient $\nabla_{\theta_k} Q_{\mu_{\theta_k}}(s, \mu_{\theta_k}(s))$.
- Each state suggests a different direction of policy improvement
- These may be averaged together by taking an expectation with respect to the state distribution $d_{\mu}(s)$

$$\theta_{k+1} = \theta_k + \alpha \cdot \mathbb{E}_{s \sim d_{\mu_{\theta_k}}} \left[\nabla_{\theta_k} Q_{\mu_{\theta_k}}(s, \mu_{\theta_k}(s)) \right]$$

- By applying the chain rule:

$$\theta_{k+1} = \theta_k + \alpha \cdot \mathbb{E}_{s \sim d_{\mu_{\theta_k}}} \left[\nabla_{\theta_k} \mu_{\theta_k}(s) \cdot \nabla_a Q_{\mu_{\theta_k}}(s, a) |_{a=\mu_{\theta_k}(s)} \right]$$

Deterministic Policy Gradient Theorem

We now formally consider a deterministic policy $\mu_{\theta} : \mathcal{S} \rightarrow \mathcal{A}$ with parameter vector $\theta \in \mathbb{R}^n$. We define a performance objective $J(\mu_{\theta}) = \mathbb{E}[G_t | \mu]$, and define probability distribution $p(s \rightarrow s', t, \mu)$ and discounted state distribution $d_{\mu}(s)$ analogously to the stochastic case.

$$J(\mu_{\theta}) = \int_{\mathcal{S}} d_{\mu}(s) \cdot r(s, \mu_{\theta}(s)) ds = \mathbb{E}_{s \sim d_{\mu}} [r(s, \mu_{\theta}(s))]$$

Theorem

Deterministic Policy Gradient

$$\begin{aligned} \nabla_{\theta} J(\mu_{\theta}) &= \int_{\mathcal{S}} d_{\mu}(s) \nabla_{\theta} \mu_{\theta}(s) \nabla_a Q_{\mu}(s, a) |_{a=\mu_{\theta}(s)} ds \\ &= \mathbb{E}_{s \sim d_{\mu}} [\nabla_{\theta} \mu_{\theta}(s) \nabla_a Q_{\mu}(s, a) |_{a=\mu_{\theta}(s)}] \end{aligned}$$

Proof: Deterministic Policy Gradient Theorem

$$\begin{aligned}\nabla_{\boldsymbol{\theta}} V_{\mu_{\boldsymbol{\theta}}}(s) &= \nabla_{\boldsymbol{\theta}} Q_{\mu_{\boldsymbol{\theta}}}(s, \mu_{\boldsymbol{\theta}}(s)) \\&= \nabla_{\boldsymbol{\theta}} \left(r(s, \mu_{\boldsymbol{\theta}}(s)) + \int_s \gamma p(s'|s, \mu_{\boldsymbol{\theta}}(s)) V_{\mu_{\boldsymbol{\theta}}}(s') ds' \right) \\&= \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s) \nabla_a r(s, a)|_{a=\mu_{\boldsymbol{\theta}}(s)} + \nabla_{\boldsymbol{\theta}} \int_s \gamma p(s'|s, \mu_{\boldsymbol{\theta}}(s)) V_{\mu_{\boldsymbol{\theta}}}(s') ds' \\&= \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s) \nabla_a r(s, a)|_{a=\mu_{\boldsymbol{\theta}}(s)} \\&\quad + \int_s \gamma \left(p(s'|s, \mu_{\boldsymbol{\theta}}(s)) \nabla_{\boldsymbol{\theta}} V_{\mu_{\boldsymbol{\theta}}}(s') + \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s) \nabla_a p(s'|s, a)|_{a=\mu_{\boldsymbol{\theta}}(s)} V_{\mu_{\boldsymbol{\theta}}}(s') \right) ds' \\&= \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s) \nabla_a \left(r(s, a) + \int_s \gamma p(s'|s, \mu_{\boldsymbol{\theta}}(s)) V_{\mu_{\boldsymbol{\theta}}}(s') \right) |_{a=\mu_{\boldsymbol{\theta}}(s)} \\&\quad + \int_s \gamma p(s'|s, \mu_{\boldsymbol{\theta}}(s)) \nabla_{\boldsymbol{\theta}} V_{\mu_{\boldsymbol{\theta}}}(s') ds'\end{aligned}$$

Proof: Deterministic Policy Gradient Theorem

(Continue)

$$\begin{aligned}
 &= \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s) \nabla_a Q_{\mu_{\boldsymbol{\theta}}}(s, a)|_{a=\mu_{\boldsymbol{\theta}}(s)} + \int_s \gamma p(s \rightarrow s', 1, \mu_{\boldsymbol{\theta}}) \nabla_{\boldsymbol{\theta}} V_{\mu_{\boldsymbol{\theta}}}(s') ds' \\
 &= \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s) \nabla_a Q_{\mu_{\boldsymbol{\theta}}}(s, a)|_{a=\mu_{\boldsymbol{\theta}}(s)} + \int_s \gamma p(s \rightarrow s', 1, \mu_{\boldsymbol{\theta}}) \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s') \nabla_a Q_{\mu_{\boldsymbol{\theta}}}(s', a)|_{a=\mu_{\boldsymbol{\theta}}(s')} ds' \\
 &\quad + \int_s \gamma p(s \rightarrow s', 1, \mu_{\boldsymbol{\theta}}) \int_s \gamma p(s' \rightarrow s'', 1, \mu_{\boldsymbol{\theta}}) \nabla_{\boldsymbol{\theta}} V_{\mu_{\boldsymbol{\theta}}}(s'') ds'' ds' \\
 &= \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s) \nabla_a Q_{\mu_{\boldsymbol{\theta}}}(s, a)|_{a=\mu_{\boldsymbol{\theta}}(s)} + \int_s \gamma p(s \rightarrow s', 1, \mu_{\boldsymbol{\theta}}) \nabla_{\boldsymbol{\theta}} \mu_{\boldsymbol{\theta}}(s') \nabla_a Q_{\mu_{\boldsymbol{\theta}}}(s', a)|_{a=\mu_{\boldsymbol{\theta}}(s')} ds' \\
 &\quad + \int_s \gamma^2 p(s \rightarrow s', 2, \mu_{\boldsymbol{\theta}}) \nabla_{\boldsymbol{\theta}} V_{\mu_{\boldsymbol{\theta}}}(s') ds'
 \end{aligned}$$

Proof: Deterministic Policy Gradient Theorem

(Continue), and we can get:

$$\nabla_{\theta} V_{\mu_{\theta}}(s) = \int_s \sum_{t=0}^{\infty} \gamma^t p(s \rightarrow s', t, \mu_{\theta}) \nabla_{\theta} \mu_{\theta}(s') \nabla_a Q_{\mu_{\theta}}(s', a)|_{a=\mu_{\theta}(s')} ds'$$

Thus (overlook γ),

$$\begin{aligned} \nabla_{\theta} J(\mu_{\theta}) &= \nabla_{\theta} \int_s d_0(s) V_{\mu_{\theta}}(s) ds \\ &= \int_s d_0(s) \nabla_{\mu_{\theta}}(s) ds \\ &= \int_s \int_s \sum_{t=0}^{\infty} d_0(s) p(s \rightarrow s', t, \mu_{\theta}) \nabla_{\theta} \mu_{\theta}(s') \nabla_a Q_{\mu_{\theta}}(s', a)|_{a=\mu_{\theta}(s')} ds' ds \\ &= \int_s d_{\mu_{\theta}}(s) \nabla_{\theta} \mu_{\theta}(s) \nabla_{\theta} Q_{\mu_{\theta}}(s, a)|_{a=\mu_{\theta}(s)} ds \end{aligned}$$

① Discrete vs. Continuous Control

② Deterministic PG (DPG)

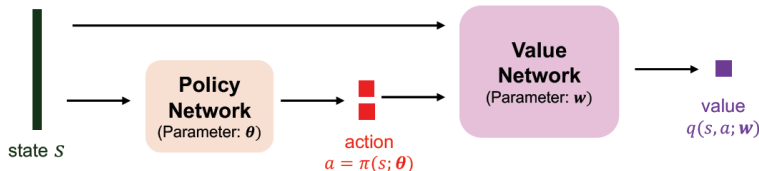
③ Deterministic PG Theorem

④ DDPG Implementation

⑤ Extension: TD3

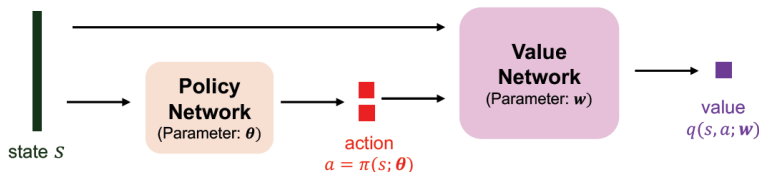
Updating Value Network by TD

- Transition: (s_t, a_t, r_t, s_{t+1})
- Value network makes a prediction for time t : $Q_t = Q(s_t, a_t; \mathbf{w})$
- Value network makes a prediction for time $t+1$: $Q_{t+1} = Q(s_{t+1}, a'_{t+1}; \mathbf{w})$, where $a'_{t+1} = \pi(s_{t+1}; \boldsymbol{\theta})$
- TD error: $\delta_t = Q_t - (r_t + \gamma \cdot Q_{t+1})$
- Update: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \delta_t \cdot \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}}$



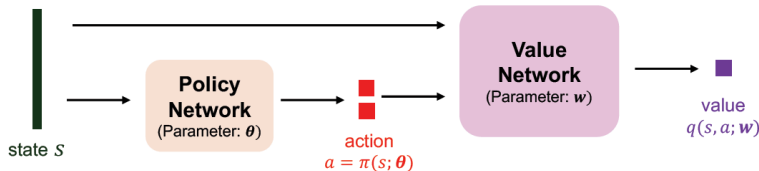
Updating Policy Network by DPG

- The critic $Q(s, a; \mathbf{w})$ evaluates how good the action a is.
- Improve $\boldsymbol{\theta}$ so that the critic believes $a = \pi(s; \boldsymbol{\theta})$ is better
- Update $\boldsymbol{\theta}$ so that $Q(s, a; \mathbf{w}) = Q(s, \pi(s; \boldsymbol{\theta}); \mathbf{w})$ increases



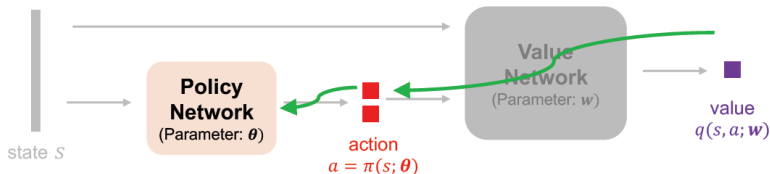
Updating Policy Network by DPG

- Goal: Increasing $Q(s, a; \mathbf{w})$, where $a = \pi(s; \boldsymbol{\theta})$
- DPG: $\mathbf{g} = \frac{\partial Q(s, \pi(s; \boldsymbol{\theta}); \mathbf{w})}{\partial \boldsymbol{\theta}} = \frac{\partial \pi(s; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot \frac{\partial Q(s, a; \mathbf{w})}{\partial a}$



Updating Policy Network by DPG

- Goal: Increasing $Q(s, a; \mathbf{w})$, where $a = \pi(s; \boldsymbol{\theta})$
- DPG: $\mathbf{g} = \frac{\partial Q(s, \pi(s; \boldsymbol{\theta}); \mathbf{w})}{\partial \boldsymbol{\theta}} = \frac{\partial \pi(s; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot \frac{\partial Q(s, a; \mathbf{w})}{\partial a}$
- Gradient ascent: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \beta \cdot \mathbf{g}$



Improvement: Using Target Networks

Bootstrapping

- Value network makes a prediction for time t : $Q_t = Q(s_t, a_t; \mathbf{w})$
- Value network makes a prediction for time $t+1$: $Q_{t+1} = Q(s_{t+1}, a'_{t+1}; \mathbf{w})$, where $a'_{t+1} = \pi(s_{t+1}; \boldsymbol{\theta})$
- TD error: $\delta_t = Q_t - (r_t + \gamma \cdot Q_{t+1})$
- **Bootstrapping**: Using ones estimate to update oneself
- **Basic idea**: Computing the TD target using different networks

Improvement: Using Target Networks

- Value network makes a prediction for time t : $Q_t = Q(s_t, a_t; \mathbf{w})$
- Target networks make a prediction for time $t + 1$:
 $Q_{t+1} = Q(s_{t+1}, a'_{t+1}; \mathbf{w}^-)$ (Target value network)
 $a'_{t+1} = \pi(s_{t+1}; \boldsymbol{\theta}^-)$ (Target policy network)
- **The same network structure, but different parameters.**

Updating Policy and Value Networks

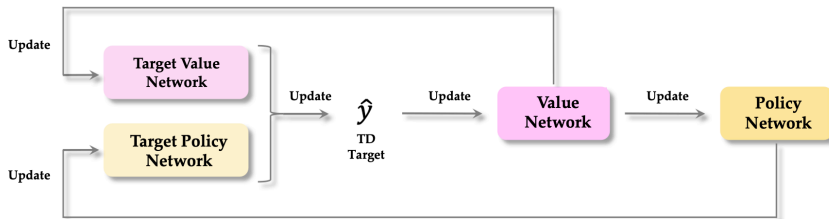
- Policy network makes a decision: $a = \pi(s; \boldsymbol{\theta})$
- Update policy network by DPG: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \beta \cdot \frac{\partial a}{\partial \boldsymbol{\theta}} \cdot \frac{\partial Q(s, a; \boldsymbol{w})}{\partial a}$
- Value network computes $q_t = Q(s, a; \boldsymbol{w})$
- Target networks, $\pi(s; \boldsymbol{\theta}^-)$ and $Q(s, a; \boldsymbol{\theta}^-)$, compute q_{t+1}
- TD error: $\delta_t = q_t - (r_t + \gamma \cdot q_{t+1})$
- Update value network by TD: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \alpha \cdot \delta_t \cdot \frac{\partial Q(s, a; \boldsymbol{w})}{\partial \boldsymbol{w}}$

Updating Target Networks

- Set a hyper-parameter $\tau \in (0, 1)$
- Update the target network by weighted averaging:

$$\mathbf{w}^- \leftarrow \tau \cdot \mathbf{w} + (1 - \tau) \cdot \mathbf{w}^-$$

$$\boldsymbol{\theta}^- \leftarrow \tau \cdot \boldsymbol{\theta} + (1 - \tau) \cdot \boldsymbol{\theta}^-$$



Deep Deterministic Policy Gradients

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .
 Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
 Initialize replay buffer R
for episode = 1, M **do**
 Initialize a random process $\mathcal{N}$ for action exploration
 Receive initial observation state s_1
 for t = 1, T **do**
 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
 Execute action a_t and observe reward r_t and observe new state s_{t+1}
 Store transition (s_t, a_t, r_t, s_{t+1}) in R
 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for
end for

Continuous Control with Deep Reinforcement Learning. In ICLR, 2016.

① Discrete vs. Continuous Control

② Deterministic PG (DPG)

③ Deterministic PG Theorem

④ DDPG Implementation

⑤ Extension: TD3

Recap: Double Tabular Q-Learning (Lec 2.2)

```
Initialize  $Q_1(s, a)$  and  $Q_2(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ , arbitrarily
Initialize  $Q_1(\text{terminal-state}, \cdot) = Q_2(\text{terminal-state}, \cdot) = 0$ 
Repeat (for each episode):
  Initialize  $S$ 
  Repeat (for each step of episode):
    Choose  $A$  from  $S$  using policy derived from  $Q_1$  and  $Q_2$  (e.g.,  $\epsilon$ -greedy in  $Q_1 + Q_2$ )
    Take action  $A$ , observe  $R, S'$ 
    With 0.5 probability:
       $Q_1(S, A) \leftarrow Q_1(S, A) + \alpha(R + \gamma Q_2(S', \arg\max_a Q_1(S', a)) - Q_1(S, A))$ 
    else:
       $Q_2(S, A) \leftarrow Q_2(S, A) + \alpha(R + \gamma Q_1(S', \arg\max_a Q_2(S', a)) - Q_2(S, A))$ 
     $S \leftarrow S'$ ;
  until  $S$  is terminal
```

Double-Q estimation $\leq \max_i \mathbb{E}[X_i] \leq \mathbb{E}[\max_i X_i]$ (single-Q estimation)

Double Q-learning. In NIPS, 2010.

TD3: Twin Delayed Deep Deterministic PG

Overestimation Issue: Solution like Double Q-learning

- Two value networks and one policy network

$$Q(s, a; \mathbf{w}_1), Q(s, a; \mathbf{w}_2), \pi(s; \boldsymbol{\theta})$$

- Each network has a corresponding target network

$$Q(s, a; \mathbf{w}_1^-), Q(s, a; \mathbf{w}_2^-), \pi(s; \boldsymbol{\theta}^-)$$

- Use the target policy network to predict the action:

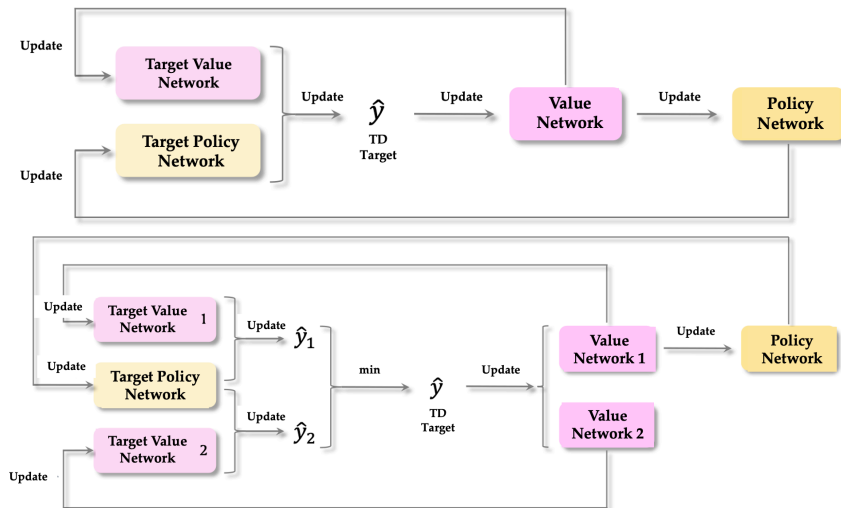
$$\hat{a}_{j+1}^- = \pi(s_{j+1}; \boldsymbol{\theta}^-)$$

- Use both target value networks to calculate corresponding TD targets:

$$\hat{y}_{j,1} = r_j + \gamma \cdot Q(s_{j+1}, \hat{a}_{j+1}^-; \mathbf{w}_1^-) \quad \hat{y}_{j,2} = r_j + \gamma \cdot Q(s_{j+1}, \hat{a}_{j+1}^-; \mathbf{w}_2^-)$$

- Take min to get the real TD target: $\hat{y}_j = \min\{\hat{y}_{j,1}, \hat{y}_{j,2}\}$

Figure Illustration: TD3



Algorithm 1 TD3

Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_ϕ
with random parameters θ_1, θ_2, ϕ

Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$

Initialize replay buffer $\mathcal{B}$

for $t = 1$ **to** T **do**

 Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$,

$\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'

 Store transition tuple (s, a, r, s') in $\mathcal{B}$

 Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$

$\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$

$y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$

 Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$

if $t \bmod d$ **then**

 Update ϕ by the deterministic policy gradient:

$\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$

 Update target networks:

$\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$

$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$

end if

end for

Extra Reading Materials

- Deterministic Policy Gradient Algorithms. In ICML, 2014.
- Continuous Control with Deep Reinforcement Learning. In ICLR, 2016.
- Addressing Function Approximation Error in Actor-Critic Methods. In ICML, 2018.